

Fix frontend test issue

08/10/2026 17:52:35

Sport-Analytics-Tool(1).zip ## Description Seventeen frontend tests fail in `beforeEach` with `TypeError:...

Thinking 🌐 Inspecting Repository ZIP Files ● Inspected frontend configuration, tests, dependencies, and...

a > > npm run test --workspace=@sport-analytics/frontend > > v24.14.0 > > npm : File C:\Program...

Thinking 🌐 Inspecting Repository History and Current Branch ● Inspected repository history, Node requirements,...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro> node -v > > npm.cmd run test --...

Thinking 🌐 Inspecting Repository Structure ● Running the missing build I'm treating the contract-build failure as a...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro> npm.cmd run test --workspace=@sport-...

Thinking 🌐 Inspecting repository files and Node compatibility ● Inspected repository files, Node compatibility, and...
so should i just make the change to be safe and not even ask him what hes using?

I wouldn't make the change yet. Ask him for node -v first. It takes him 10 seconds and gives you the evidence you...
ok, so how do i get out of his branch or whatever i did to see it from his aspect

Yes — you created a separate Git worktree, so your normal repo/branch is still untouched. You're currently here:...

i had created 108 branch, so now what?

If you already created the #108 branch in your normal repo, you don't need to create it again. Go back to the nor...

3b35f8d2-15cc-4219-a844-c33343173b15.png

Thinking ● Cleaning up worktree I'm seeing a Windows cleanup issue: the temporary worktree is still registered,...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git -C ..\Sport-Analytics-Tool-108-repro status --short fat...

That actually means the temporary worktree's Git metadata inside that folder is already gone. What's probably left...
4a46dfee-c850-42d4-912e-0646b23d33f6.png

Perfect — you're clean now. You have exactly one worktree, and you're on: fix/108-frontend-localstorage-tests So t...
they not answering, is it safe to assume its 25?

Thinking 🌐 Searching Vitest jsdom Node 25 localStorage clear is not a function issue 8757...

another person who also doesnt have errors has node 24, could i use 25, test, and then go back to 24?

Thinking 🌐 Searching the web nodejs.orggithub.com ● Searched 2 websites ✓ Worked for 15s Yes - that's a good...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> nvm version nvm : The term 'nvm' is not recognized as...

Thinking 🌐 Searching the web...

PS C:\Users\shayn\OneDrive\Desktop> cd C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool PS...

Thinking 🌐 Inspecting Repository Structure and Setup ● Inspected repository setup and investigated Node...
both pass!

Thinking ● Verifying the fix I'm ready to run the final checks now: lint, type-checking, formatting validation, and th...
give me commit line with message, also, dont i need to first add AI evidence and this transcript b4 i commit?

Thinking 🌐 Inspecting AI evidence files in repository zip ● Preparing commit guidance I'm checking the requested...
i need to format vite first, whats the command?

Thinking 🌐 Inspecting frontend package scripts ● Inspected frontend package scripts in project and ZIP archive ✓...



Sport-Analytics-Tool(1).zip

Description

Seventeen frontend tests fail in `beforeEach` with
`TypeError: window.localStorage.clear is not a function`.

Affected files:

- `apps/frontend/src/App.test.tsx` (9 tests)
- `apps/frontend/src/pages/PublicBrowsePages.test.tsx` (8 tests)

Both call `window.localStorage.clear()` in `beforeEach`. Something in the test environment provides a `localStorage` object without a `clear` method.

Reproduction

```
npm run test --workspace=@sport-analytics/frontend
```

Observed on `main` at commit 1f9a3aa on 10 August 2026.

Motivation

`npm run check` is the only merge gate the project has, because the repository has no CI runners. While it fails on `main` for reasons unrelated to any given branch, members cannot use it to judge their own work, and a real failure will be mistaken for this one.

Acceptance Criteria

- [] All seventeen tests pass
- [] `npm run test --workspace=@sport-analytics/frontend` passes on a clean checkout
- [] The cause is identified rather than worked around by removing the `clear()` calls

Evidence

Test output before and after, and the linked Pull Request.

AI Declaration: The preceding issue was generated with the assistance of Claude-Web[Claude Opus 5].

heres the issue i need to fix

issue #108

ive given the zip fileof the repo, help

heres the message he sent before making the issu

Heads up — npm run check is failing on main at the moment, two separate things, neither of them anyone's branch:

1. Backend typecheck fails with "Cannot find module '@sport-analytics/contracts'" unless you build contracts first. check runs typecheck before build so a fresh clone always fails here. Workaround for now:

```
npm run build --workspace=@sport-analytics/contracts
```

2. 17 frontend tests fail in beforeEach with "window.localStorage.clear is not a function" — App.test.tsx and PublicBrowsePages.test.tsx. Something in the test setup is stubbing localStorage without a clear method.

Flagging because we've got no CI runners, so check is the only gate we have. If it's red by default we'll all start ignoring it and something real will slip through.

Who owns the frontend auth tests? And whoever's on public-read, the contracts ordering is probably a one-line fix in the check script.

8/10/2026, 4:52:20 PM



Thinking

Inspecting Repository ZIP Files

- Inspected frontend configuration, tests, dependencies, and localStorage authentication setup

Searching the web

github.com developer.mozilla.org nodejs.org w3schools.com blog.logrocket.com en.wikipedia.org nodered.org

- Searched 7 websites

Inspecting Vitest Global Population Internals

- Fixing localStorage tests

I'm tracing why Node 25's existing `localStorage` interferes with jsdom. Pinning Node isn't sufficient for clean environments, so I'm considering a Vitest setup that explicitly restores jsdom's storage object.

- Fixed localStorage tests

- Inspected Vitest global population and localStorage handling

Searching the web

- Searching github.com

Inspecting Node Compatibility Guidance

- Inspected Node.js compatibility guidance and build package metadata

Searching the web

- Searching the web

- Confirming the root cause

I'm narrowing this to Node 25's preexisting `localStorage`, which prevents Vitest's jsdom environment from installing its own storage. I'm also separating that diagnosis from sandbox limitations caused by platform-specific dependencies.

- Confirmed the root cause

Testing Node Web Storage Flag Behavior

- Inspected project configurations, documentation, tooling, and validated package builds

✓ Worked for 10m 30s

Yes - I went through the repo ZIP, and **Issue #108 is a small, contained frontend-test-environment fix**. You should not change the 17 tests or remove the `clear()` calls.

The important finding is that **nothing in your repository is manually stubbing** `localStorage`.

`apps/frontend/src/test/setup.ts` currently only installs the Testing Library matchers, while both failing suites legitimately use the

browser Storage API. Your frontend is using Vitest 2.1.x with jsdom. The failure matches a known Vitest/Node interaction: Node 25 enabled its own Web Storage global by default, and Vitest's jsdom environment can then leave that Node `localStorage` in place instead of exposing jsdom's proper `Storage`. In Node 25, that object can be an empty object, hence `clear` is not a function. (GitHub)

So I would fix **#108 only**, and leave the contracts/check-order issue to its separate branch.

1. Make your branch

From the latest `main`:

```
git switch main
git pull origin main
git switch -c fix/108-frontend-localstorage-tests
```

Before changing anything, run:

```
node -v
npm run test --workspace=@sport-analytics/frontend
```

If the machine reporting the problem is on Node 25.x, that essentially confirms the upstream cause. Node's own documentation records that Node 25 exposes Web Storage by default and, without a storage file, `localStorage` can be an empty object. (Node.js) Your repo ZIP is actually newer than the issue's reproduction commit - the ZIP's `main` is at 4742e4f, whereas #108 says it was first observed at 1f9a3aa - but the relevant Vitest/jsdom setup is still the same.

2. Change only `apps/frontend/src/test/setup.ts`

Replace the current:

```
import * as matchers from '@testing-library/jest-dom/matchers';
import { expect } from 'vitest';

expect.extend(matchers);
```

with:

```
import * as matchers from '@testing-library/jest-dom/matchers';
import { afterAll, expect } from 'vitest';

expect.extend(matchers);

type VitestJsdomGlobal = typeof globalThis & {
  jsdom?: { window: Window };
};

const originalLocalStorageDescriptor = Object.getOwnPropertyDescriptor(
  globalThis,
  'localStorage',
);

const jsdomLocalStorage = (globalThis as VitestJsdomGlobal).jsdom?.window.localStorage;

function hasUsableLocalStorage(): boolean {
  try {
    return typeof globalThis.localStorage.clear === 'function';
  } catch {
    return false;
  }
}

// Node's Web Storage global can prevent Vitest from exposing jsdom's
```

```
// browser-compatible Storage implementation. Restore jsdom's implementation
// when the inherited global is unusable.
if (jsdomLocalStorage && !hasUsableLocalStorage()) {
  Object.defineProperty(globalThis, 'localStorage', {
    configurable: true,
    value: jsdomLocalStorage,
    writable: true,
  });
}

afterAll(() => {
  if (originalLocalStorageDescriptor) {
    Object.defineProperty(
      globalThis,
      'localStorage',
      originalLocalStorageDescriptor,
    );
  } else {
    Reflect.deleteProperty(globalThis, 'localStorage');
  }
});
```

Why I prefer this fix

This isn't faking the test result or deleting the `clear()` calls. It restores **jsdom's real browser-style** `localStorage` **implementation** when Vitest has inherited an unusable Node global.

That means these remain proper tests:

```
window.localStorage.clear();
window.localStorage.getItem(...);
window.localStorage.setItem(...);
```

and your actual application code in `theme.ts` isn't changed merely to accommodate a test-runner/runtime collision.

I would **not** change the tests to something like:

```
window.localStorage = {
  getItem: vi.fn(),
  setItem: vi.fn(),
};
```

because that would just replace the broken environment with another incomplete fake. I also wouldn't upgrade Vitest/Vite on this bug-fix branch; that creates considerably more dependency risk than #108 warrants.

3. Run the acceptance checks

First:

```
npm run test --workspace=@sport-analytics/frontend
```

You want all frontend tests green, including the 17 that previously died in `beforeEach`.

Then:

```
npm run lint --workspace=@sport-analytics/frontend
npm run typecheck --workspace=@sport-analytics/frontend
```

Remember the other known `main` problem: if typechecking complains that it cannot find `@sport-analytics/contracts`, build `contracts` first:

```
npm run build --workspace=@sport-analytics/contracts
npm run typecheck --workspace=@sport-analytics/frontend
```

That contracts-ordering issue is **not part of #108**, so don't fix the root `check` script on this branch.

For evidence for #108, save the **before output showing the 17 failures** and the **after output showing the frontend suite passing**.

4. Check your diff

```
git status
git diff
git diff --check
```

For this issue, your functional diff should ideally be just:

```
apps/frontend/src/test/setup.ts
```

plus your required AI evidence files.

5. Commit

Because we're generating code together here, the course AI policy requires the code-generation attribution in the commit. `ai_policy_20_jul.pdf` Your repo README already declares `ChatGPT-Web[GPT-5.6 Sol]`, so you don't need to add the tool to README again.

Use:

```
git add apps/frontend/src/test/setup.ts
git commit
```

Commit message:

```
fix(frontend): restore jsdom localStorage in tests
```

Refs #108

Assisted-by: ChatGPT-Web[GPT-5.6 Sol]

Then push:

```
git push -u origin fix/108-frontend-localstorage-tests
```

For the PR, I'd make the core explanation something like: **Node's Web Storage global can shadow jsdom's Storage under Vitest, leaving `window.localStorage` without the expected browser methods. The test setup now restores jsdom's Storage implementation when the inherited global is unusable. No test cleanup calls were removed.**

That directly satisfies the important acceptance criterion: **the cause was identified rather than hidden.** ([GitHub](#))

One note from my side: I couldn't faithfully execute the frontend suite inside my sandbox from your ZIP because the ZIP contains Windows `node_modules` (`rollup-win32...`) while my execution environment is Linux. So I inspected the source/configuration and matched it against the upstream Vitest/Node bug rather than pretending I had a valid local green run. Your machine's before/after run will be the authoritative evidence.

If you paste me the output of `node -v` and the first ~20 lines of the failing frontend test run, I can confirm the diagnosis before you make the edit.

Sources

[Node v25 breaks tests with Web Storage API · Issue #8757 · vitest-dev/vitest · GitHub](#) github.com



a

```
> > npm run test --workspace=@sport-analytics/frontend
> > v24.14.0
> > npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running
scripts is disabled on this system. For
> > more information, see about\_Execution\_Policies at
https://go.microsoft.com/fwlink/?LinkID=135170.
> > At line:2 char:1
```

```
- npm run test --workspace=@sport-analytics/frontend
-
...
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
...
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run test --
workspace=@sport-analytics/frontend
```

```
> @sport-analytics/frontend\@0.1.0 test
> vitest run
```

```
RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/frontend
```

```
✓ src/App.test.tsx (13) 510ms
✓ src/api/client.test.ts (4)
✓ src/api/public-read.test.ts (3)
✓ src/pages/PublicBrowsePages.test.tsx (8) 587ms
✓ src/features/auth/AuthProvider.test.tsx (4)
```

```
Test Files 5 passed (5)
Tests 32 passed (32)
Start at 17:03:58
Duration 2.38s (transform 383ms, setup 524ms, collect 1.18s, tests 1.29s,
environment 3.29s, prepare 960ms)
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/10/2026, 5:04:35 PM



Thinking

🌐 Inspecting Repository History and Current Branch

- Inspected repository history, Node requirements, tests, storage usage, and Vitest configuration

🌐 Searching the web

github.com nodejs.org developer.mozilla.org en.wikipedia.org nodered.org

- Interpreting the test results

I'm treating this as an environment-specific failure, not an immediate code defect. The PowerShell error is separate, while tests passing on current `main` suggest reproducing the exact referenced commit in a clean worktree before changing implementation.

- Interpreted the test results

- Searched 5 websites

🌐 Checking current Node.js environment

- **Reproducing the report**

I'm checking the exact reported commit before editing code. If the failure disappears there too, I'll treat Node-version differences as the likely cause and ask for the reporter's runtime version before proposing a fix.

- **Reproduced the report**

- **Checked the current Node.js environment**



- **Searching the web**

github.com

nodejs.org

- **Searched 2 websites**

✓ Worked for 1m 57s

Good - **this is important. Do not change any code yet.**

Your result tells us:

Node: v24.14.0

Frontend tests: 32 passed / 32

So on **Node 24**, the bug does not occur. The first `npm` error is just Windows PowerShell refusing to execute `npm.ps1`; using `npm.cmd` is perfectly fine for your work.

More importantly, I checked the ZIP: the two `window.localStorage.clear()` calls are **still there**, and `apps/frontend/src/test/setup.ts` has not been changed to fix them. So the tests haven't magically been "fixed" on `main`.

This strongly points to **the Node version being the actual cause**.

Node 25 changed Web Storage: `localStorage` became enabled by default, and if no storage file is supplied it can expose an empty object instead of the jsdom storage object. ([Node.js](#)) Vitest has an upstream issue describing **exactly this symptom**: tests using jsdom work on Node 24 but fail on Node 25 because methods such as `localStorage.getItem` / `clear` are missing. ([GitHub](#))

Do this next

First check exactly what commit you're currently testing:

```
git rev-parse --short HEAD
```

Send me the result.

Then we're going to test the **exact commit named in #108**, `1f9a3aa`, without messing with your current branch.

From the repo:

```
git worktree add ..\Sport-Analytics-Tool-108-repro 1f9a3aa
```

Then:

```
cd ..\Sport-Analytics-Tool-108-repro
```

Install from the lockfile so this is genuinely close to a clean checkout:

```
npm.cmd ci
```

Then:

```
node -v
npm.cmd run test --workspace=@sport-analytics/frontend
```

What I expect

On your **Node v24.14.0**, I expect the 17 tests to pass even at `1f9a3aa`.

If they do, we've established something much better than blindly changing `setup.ts`:

Issue #108 is environment/version-specific, not caused by a repository stub of `localStorage`.

The upstream Vitest reproduction specifically reports Node 25 failing and Node 24 succeeding. ([GitHub](#))

At that point, the next thing we need from the teammate who opened #108 is simply:

```
node -v
```

If theirs says **v25.x**, we've basically nailed the root cause.

And importantly: **don't use the `setup.ts` code I gave you in my previous message yet.** Your new evidence means we should prove the runtime cause first and then make the smallest correct fix for #108, rather than introducing a speculative workaround.

Sources

[Global objects | Node.js v26.7.0 Documentation](#) [nodejs.org](#)

[Node v25 breaks tests with Web Storage API · Issue #8757 · vitest-dev/vitest · GitHub](#) [github.com](#)

8/10/2026, 5:04:38 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro> node -v
```

```
> > npm.cmd run test --workspace=@sport-analytics/frontend  
> > v24.14.0
```

```
> @sport-analytics/frontend\@0.1.0 test  
> vitest run
```

```
RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool-108-  
repro/apps/frontend
```

```
› src/App.test.tsx (0)  
✓ src/api/client.test.ts (4)  
› src/api/public-read.test.ts (0)  
› src/pages/PublicBrowsePages.test.tsx (0)  
✓ src/features/auth/AuthProvider.test.tsx (4)
```

Failed Suites 3

```
FAIL src/App.test.tsx [ src/App.test.tsx ]  
FAIL src/api/public-read.test.ts [ src/api/public-read.test.ts ]  
FAIL src/pages/PublicBrowsePages.test.tsx [ src/pages/PublicBrowsePages.test.tsx ]  
Error: Failed to resolve entry for package "@sport-analytics/contracts". The package  
may have incorrect main/module/exports specified in its package.json.  
Plugin: vite:import-analysis  
File: C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool-108-  
repro/apps/frontend/src/api/public-read.ts  
› packageEntryFailure ../../node_modules/vite/dist/node/chunks/dep-  
BK3b2jBa.js:46638:15  
› resolvePackageEntry ../../node_modules/vite/dist/node/chunks/dep-  
BK3b2jBa.js:46635:3  
› tryNodeResolve ../../node_modules/vite/dist/node/chunks/dep-  
BK3b2jBa.js:46451:16  
› ResolvedContext.resolve ../../node_modules/vite/dist/node/chunks/dep-  
BK3b2jBa.js:46201:19  
› PluginContainer.resolve ../../node_modules/vite/dist/node/chunks/dep-  
BK3b2jBa.js:49018:17  
› TransformPluginContext.resolve ../../node_modules/vite/dist/node/chunks/dep-  
BK3b2jBa.js:49178:15  
› normalizeUrl ../../node_modules/vite/dist/node/chunks/dep-BK3b2jBa.js:64300:26  
› ../../node_modules/vite/dist/node/chunks/dep-BK3b2jBa.js:64439:39
```

Test Files 3 failed | 2 passed (5)

Tests 8 passed (8)

Start at 17:09:01

Duration 6.12s (transform 361ms, setup 2.78s, collect 805ms, tests 156ms,
environment 17.51s, prepare 2.33s)

npm error Lifecycle script `test` failed with error:

npm error code 1

npm error path C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-
repro\apps\frontend

npm error workspace @sport-analytics/frontend\@0.1.0

npm error location C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-
repro\apps\frontend

npm error command failed

```
npm error command C:\Windows\system32\cmd.exe /d /s /c vitest run
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro>
```

8/10/2026, 5:09:23 PM



Thinking

Inspecting Repository Structure

- **Running the missing build**

I'm treating the contract-build failure as a prerequisite rather than evidence of the localStorage bug. I'd build the contracts first, rerun the frontend tests, and use the result to distinguish an environment mismatch from a reproducible regression.

- **Ran the missing build**

- **Inspected repository structure, commit setup, and test changes**

✓ Worked for 1m 2s

Yep — this is useful. **You have not reached Issue #108 yet.**

This failure is exactly the **other problem** your teammate mentioned:

```
Failed to resolve entry for package "@sport-analytics/contracts"
```

Vitest is failing while loading the affected test files, so `beforeEach()` never runs. That's why you see:

```
src/App.test.tsx (0)
src/api/public-read.test.ts (0)
src/pages/PublicBrowsePages.test.tsx (0)
```

So stay in the `Sport-Analytics-Tool-108-repro` worktree and run the workaround your teammate gave:

```
npm.cmd run build --workspace=@sport-analytics/contracts
```

You should hopefully get a successful contracts build.

Then immediately run:

```
npm.cmd run test --workspace=@sport-analytics/frontend
```

What this test will tell us

There are two possible outcomes.

Outcome A — you now get the 17 `localStorage.clear` failures

Excellent. We've reproduced #108, and we can investigate the actual cause on your Node 24 machine.

Outcome B — all 32 tests pass

Also very useful. That would mean:

```
commit 1f9a3aa
+ clean npm ci
+ contracts built
+ Node v24.14.0
= no localStorage bug
```

That would strongly suggest the person who reported #108 had a different runtime/environment, and we should **not modify the repository until we identify that difference**.

Also, this run has independently confirmed your teammate's first claim:

A clean checkout cannot resolve @sport-analytics/contracts until contracts are built.

So their contracts-ordering bug is definitely real.

For now, just run these two:

```
npm.cmd run build --workspace=@sport-analytics/contracts  
npm.cmd run test --workspace=@sport-analytics/frontend
```

Send me the full result of the **second command**. That's the one that will tell us what #108 actually is.

8/10/2026, 5:09:25 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro> npm.cmd run test --workspace=@sport-analytics/frontend
```

```
> @sport-analytics/frontend@0.1.0 test
> vitest run
```

```
RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool-108-repro/apps/frontend
```

```
✓ src/App.test.tsx (9) 396ms
✓ src/api/client.test.ts (4)
✓ src/api/public-read.test.ts (3)
✓ src/pages/PublicBrowsePages.test.tsx (8) 570ms
✓ src/features/auth/AuthProvider.test.tsx (4)
```

```
Test Files 5 passed (5)
Tests 28 passed (28)
Start at 17:11:47
Duration 5.69s (transform 381ms, setup 2.73s, collect 2.29s, tests 1.14s, environment 16.02s, prepare 1.21s)
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro>
```

8/10/2026, 5:12:37 PM



Thinking

🌐 Inspecting repository files and Node compatibility

- Inspected repository files, Node compatibility, and Vitest configuration

🌐 Searching the web

developer.mozilla.org vitest.dev nodejs.org w3schools.com main.vitest.dev blog.logrocket.com en.wikipedia.org github.com
deno.com nodered.org cn.vitest.dev

- Searched 11 websites

🌐 Checking documented Node.js version requirements

- Evaluating test fixes

I'm comparing two approaches: constrain supported Node versions or change the test command to disable web storage. Version constraints only warn, so upgrading the test tooling appears more robust than relying on environment-specific flags.

- Evaluated test fixes

🌐 Checking Node and npm versions

- Selecting the practical fix

I'm treating this as a Node 25 compatibility issue, not a repository regression. Node 24 passes, while the project currently permits newer runtimes. I'd document and enforce the supported LTS range, then consider a separate tooling upgrade.

- Selected the practical fix

🌐 Searching Frontend localStorage Usage

- Checked Node requirements, versions, and frontend localStorage usage

✓ Worked for 1m 47s

Great - this narrows it down a lot.

You have now shown that at the exact reproduction commit `1f9a3aa`, after building `@sport-analytics/contracts`, **all 28 frontend tests pass on Node v24.14.0**. So Issue #108 is not caused by something in those tests randomly replacing `localStorage`.

The upstream Vitest bug matches this almost perfectly: Node 25 enabled its own Web Storage global, which can interfere with Vitest's `jsdom` environment; the upstream reproduction explicitly says the same test fails on Node 25 and passes on Node 24. ([GitHub](#))

Your repo also currently says:

```
"engines": {
  "node": ">=20"
}
```

so Node 25 is technically allowed by the repository, even though your CI workflow uses Node 20 and your deployment files use Node 22.

What I would do now

Before touching code, send the person who opened #108 this:

*Can you please run `node -v` on the machine where you got the 17 `localStorage` failures?
I tested the exact reproduction commit `1f9a3aa` on Node v24.14.0. After building `contracts`, all 28 frontend tests pass. There is an upstream Vitest issue where Node 25 causes exactly `localStorage` methods such as `clear()` / `getItem()` to disappear under `jsdom`, while Node 24 passes.*

If they reply with **v25.x**, we have the root cause.

And Node 25 is now an EOL release; Node currently lists Node 24 and Node 22 as LTS lines. ([Node.js](#))

But we should still fix #108

I would **not close the issue as “works on my machine.”** Your repository says Node `>=20`, so somebody using Node 25 is currently doing something the repo claims is supported.

There are two possible fixes. For this project I recommend the smaller one: make the Vitest setup explicitly use `jsdom`'s browser `localStorage` when the runtime has supplied an unusable one.

That keeps the existing:

```
window.localStorage.clear();
```

exactly as it is and addresses the actual environment collision.

Create your branch from current `main`, not from the reproduction worktree:

```
cd C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool

git switch main
git pull origin main
git switch -c fix/108-localstorage-test-environment
```

Then open:

```
apps/frontend/src/test/setup.ts
```

It currently contains:

```
import * as matchers from '@testing-library/jest-dom/matchers';
import { expect } from 'vitest';

expect.extend(matchers);
```

Change it to:

```
import * as matchers from '@testing-library/jest-dom/matchers';
import { expect } from 'vitest';

expect.extend(matchers);

type JsdomGlobal = typeof globalThis & {
  jsdom?: {
    window: Window;
  };
};

const jsdomWindow = (globalThis as JsdomGlobal).jsdom?.window;

if (
  jsdomWindow &&
  typeof window.localStorage.clear !== 'function'
) {
  Object.defineProperty(window, 'localStorage', {
    configurable: true,
    value: jsdomWindow.localStorage,
  });
}
```

Vitest documents that its `jsdom` environment exposes the underlying `jsdom` instance globally, so this uses the actual `jsdom` `Storage` object rather than inventing a fake one. ([Vitest](#))

Then test it on your Node 24 machine

First build contracts because that is the separate existing `main` problem:

```
npm.cmd run build --workspace=@sport-analytics/contracts
```

Then:

```
npm.cmd run test --workspace=@sport-analytics/frontend
```

You should still get:

```
Test Files  5 passed (5)
Tests      32 passed (32)
```

Then run:

```
npm.cmd run lint --workspace=@sport-analytics/frontend
npm.cmd run typecheck --workspace=@sport-analytics/frontend
```

Don't commit yet. **Send me the output of those three commands.**

Ideally, after that we also get the reporter to test your branch on their Node 25 machine. That gives you very strong evidence for the PR:

```
Before:
Node 25 → 17 localStorage failures

After:
Node 25 → all frontend tests pass

Regression:
Node 24 → all frontend tests still pass
```

That is a much stronger #108 fix than simply changing or removing the `clear()` calls.

Also keep the AI attribution in mind: because we're generating code for the fix, your eventual commit needs an `Assisted-by:` line under the course policy. `ai_policy_20_jul.pdf`

Sources

[Node v25 breaks tests with Web Storage API · Issue #8757 · vitest-dev/vitest · GitHub](#) github.com

[Node.js Releases](#) nodejs.org

[environment | Config](#) vitest.dev

8/10/2026, 5:12:39 PM



so should i just make the change to be safe and not even ask him what hes using?

8/10/2026, 5:16:38 PM



I wouldn't make the change yet. **Ask him for** `node -v` **first**. It takes him 10 seconds and gives you the evidence you need for Issue #108.

Right now you have proven:

```
Commit 1f9a3aa
Node v24.14.0
contracts built
→ 28/28 frontend tests pass
```

So if you now add a `localStorage` workaround without knowing what environment produced the failure, your PR could reasonably be challenged with: "How do we know this fixes the actual cause?"

Send him:

Hey, I'm looking at #108. I tested the exact reproduction commit `1f9a3aa` on Node v24.14.0 and, after building contracts, all 28 frontend tests pass. Can you please send me the output of `node -v` from the machine where you got the 17 `localStorage.clear()` failures?

If he says **Node 25**, then great — we have a clean root cause and can make the fix confidently.

If he says **Node 24**, then the Node-25 theory is wrong and we'd have saved ourselves from committing the wrong fix.

Also, the issue explicitly says:

"The cause is identified rather than worked around"

So asking him is actually the **better way to satisfy the acceptance criteria**.

You can continue setting up your `fix/108-...` branch meanwhile, but I would wait for that one answer before changing `setup.ts`.

8/10/2026, 5:16:48 PM



ok, so how do i get out of his branch or whatever i did to see it from his aspect

8/10/2026, 5:17:57 PM



Yes — you created a **separate Git worktree**, so your normal repo/branch is still untouched.

You're currently here:

```
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro
```

That folder was just a temporary copy of commit `1f9a3aa` so we could reproduce his issue.

Go back to your normal repo with:

```
cd C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool
```

Then check where you are:

```
git status
git branch --show-current
```

You should see your normal branch, probably `main` if you hadn't created the `#108` branch yet.

Then delete the temporary reproduction folder

First, while in your **normal repo**, check the worktrees:

```
git worktree list
```

You should see something roughly like:

```
C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool          4742e4f [main]
C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool-108-repro 1f9a3aa (detached HEAD)
```

Before removing it, just make sure you didn't accidentally edit anything there:

```
git -C ../Sport-Analytics-Tool-108-repro status
```

If it says something like:

```
HEAD detached at 1f9a3aa
nothing to commit, working tree clean
```

remove it:

```
git worktree remove ../Sport-Analytics-Tool-108-repro
```

Then:

```
git worktree list
```

and only your normal repo should remain.

You did not switch your real repo onto his commit. We made a totally separate temporary folder specifically so we could inspect `1f9a3aa` without disturbing your actual work.

Once you're back, send me the output of:

```
git status  
git branch --show-current
```

and I'll tell you exactly how to start your #108 branch.

8/10/2026, 5:18:07 PM



i had created 108 branch, so now what?

8/10/2026, 5:19:16 PM



If you already created the #108 branch in your normal repo, you don't need to create it again.

Go back to the normal repo:

```
cd C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool
```

Then check:

```
git branch --show-current  
git status
```

If it shows something like:

```
fix/108-localstorage-test-environment
```

or whatever name you gave the branch, you're exactly where you need to be.

The temporary `Sport-Analytics-Tool-108-repro` folder was separate and did not change your #108 branch.

If you still want to remove the temporary reproduction worktree, from your normal repo run:

```
git worktree list
```

Then, assuming the temporary one is listed and you made no edits there:

```
git worktree remove ..\Sport-Analytics-Tool-108-repro
```

After that, stay on your existing #108 branch and **don't make the code change yet**. We're waiting for the other person's `node -v` so we can confirm what actually caused the failure.

So your immediate next move is just:

```
cd C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool  
git branch --show-current  
git status
```

Send me that output and I'll confirm everything is clean.

8/10/2026, 5:19:24 PM



3b35f8d2-15cc-4219-a844-c33...

8/10/2026, 5:20:18 PM



Thinking

- **Cleaning up worktree**

I'm seeing a Windows cleanup issue: the temporary worktree is still registered, likely because files remain locked or untracked. I'd close anything using that folder, then retry removal with force and prune stale metadata if needed.

- **Cleaned up worktree**

✓ Worked for 12s

You're in the right place now. Your real repo is on:

```
fix/108-frontend-localstorage-tests
```

and the temporary reproduction worktree is still hanging around separately. Nothing is wrong with your #108 branch.

The delete failed because Windows/OneDrive is probably holding something open inside the temporary folder — most likely from the `npm ci` / `node_modules` we created there.

First, from where you are now, check that there's nothing important modified in the temporary copy:

```
git -C ..\Sport-Analytics-Tool-108-repro status --short
```

If that gives **no output**, the temporary worktree is clean and safe to delete.

Then try:

```
git worktree remove --force ..\Sport-Analytics-Tool-108-repro
```

After that:

```
git worktree list
```

Ideally you should now only see:

```
C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool 4742e4f [fix/108-frontend-localstorage-tests]
```

If it still says "Directory not empty"

Don't worry. First close any VS Code windows, File Explorer windows, or terminals that are inside:

```
Sport-Analytics-Tool-108-repro
```

Then run:

```
Remove-Item -Recurse -Force ..\Sport-Analytics-Tool-108-repro
git worktree prune
git worktree list
```

If PowerShell itself refuses to delete `node_modules`, use Windows' `rmdir` instead:

```
cmd /c rmdir /s /q "C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro"
git worktree prune
```

```
git worktree list
```

Do not delete Sport-Analytics-Tool — only the folder ending in -108-repro .

And your actual #108 branch is already correct, so once that temporary folder is gone, stay exactly where you are. No branch switching needed.

8/10/2026, 5:20:30 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git -C ..\Sport-Analytics-Tool-108-repro status --short
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/10/2026, 5:21:31 PM



That actually means the temporary worktree's Git metadata inside that folder is already gone. What's probably left is just ordinary files like `node_modules`, which is why Git can't treat it as a repo anymore.

Run:

```
git worktree list
```

If `Sport-Analytics-Tool-108-repro` is still listed, clean up the stale worktree reference with:

```
git worktree prune
```

Then check again:

```
git worktree list
```

After that, delete the leftover folder itself:

```
cmd /c rmdir /s /q "C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool-108-repro"
```

Then confirm:

```
Test-Path ..\Sport-Analytics-Tool-108-repro
git worktree list
```

You want:

```
False
```

and only your real repo listed, on:

```
[fix/108-frontend-localstorage-tests]
```

Your actual #108 branch is fine; this is just cleanup of the temporary reproduction folder.

8/10/2026, 5:21:40 PM



4a46dfee-c850-42d4-912e-06...

8/10/2026, 5:22:39 PM



Perfect — you're clean now.

You have exactly one worktree, and you're on:

```
fix/108-frontend-localstorage-tests
```

So the temporary reproduction copy is gone.

At this point, do **nothing to the code yet** until the person who opened #108 sends you their:

```
node -v
```

You've already established that on `v24.14.0`, the exact reported commit passes once contracts are built. The next step is just confirming whether their environment is Node 25.

8/10/2026, 5:22:48 PM





they not answering, is it safe to assume its 25?

8/10/2026, 5:32:58 PM



Thinking



Searching Vitest jsdom Node 25 localStorage clear is not a function issue 8757

[github.com](#) [zenn.dev](#) [stackoverflow.com](#) [appwrite.io](#) [vite.dev](#) [runthatline.com](#) [developer.mozilla.org](#) [reddit.com](#)
[youtube.com](#) [arxiv.org](#)

- Searched 10 websites

✓ Worked for 5s

It's a **very plausible** assumption, but I wouldn't treat it as proven.

You now have two solid pieces of evidence:

- Exact reported commit `1f9a3aa` + Node `v24.14.0` + contracts built → **28/28 tests pass**.
- Vitest has a documented Node 25 issue where jsdom/happy-dom tests can get a broken `localStorage` object with missing methods. ([GitHub](#))

So yes, Node 25 is currently the **leading explanation**. But Issue #108 specifically says the cause should be identified, so I'd avoid writing "the cause was Node 25" unless you actually reproduce it on Node 25 or get the reporter's version.

The best next step is to reproduce it yourself on Node 25. Then you don't need to wait for him at all.

If you have `nvm-windows`, we can install Node 25 temporarily, run the exact failing tests, confirm the 17 failures, then switch straight back to Node 24. That would give you definitive before/after evidence for the PR.

If you want, I can walk you through doing that safely without affecting your project branch.

Sources

[Node v25 breaks tests with Web Storage API · Issue #8757](#) [github.com](#)

8/10/2026, 5:33:00 PM



another person who also doesn't have errors has node 24, could i use 25, test, and then go back to 24?

8/10/2026, 5:35:13 PM



Thinking



Searching the web

nodejs.org

github.com

• Searched 2 websites

✓ Worked for 15s

Yes - that's a good way to confirm it yourself. You can switch to Node 25, run the test, then switch back to Node 24. Your Git branch/files won't be changed just by switching Node versions.

Since you're on Windows, the cleanest way is with **nvm-windows**. It lets you install multiple Node versions and switch between them; its docs recommend running the commands from an Administrator shell. ([GitHub](#))

First check whether you already have it:

```
nvm version
```

If that works, check your installed versions:

```
nvm list
```

Then install the final Node 25 release:

```
nvm install 25.9.0
```

Node **25.9.0** was the last Node 25 release; Node 25 is now EOL, so we're using it **only to reproduce the bug**, not as your project's normal development version. ([Node.js](#))

Switch to it:

```
nvm use 25.9.0
node -v
```

You want:

```
v25.9.0
```

Then, on your current #108 branch, run:

```
npm.cmd run build --workspace=@sport-analytics/contracts
npm.cmd run test --workspace=@sport-analytics/frontend
```

What we're hoping to see

If Node 25 produces:

```
TypeError: window.localStorage.clear is not a function
```

for those 17 tests, then you've independently reproduced #108.



That would be particularly strong evidence because Node 25 explicitly changed Web Storage to be **enabled by default**, which is the relevant runtime change here. ([Node.js](#))

Save/screenshot that output.

Then go straight back to Node 24

You currently have:

```
v24.14.0
```

so:

```
nvm use 24.14.0  
node -v
```

Confirm:

```
v24.14.0
```

Then:

```
npm.cmd run test --workspace=@sport-analytics/frontend
```

and you should be back to your current passing state.

So yes: **25 → reproduce → 24 again** is exactly what I'd do now. If `nvm version` says the command doesn't exist, tell me that before installing anything manually and I'll give you the safest Windows setup.

Sources

[coreybutler/nvm-windows: A node.js version management ...](#) github.com

[Node.js Releases](#) nodejs.org

8/10/2026, 5:35:15 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> nvm version
nvm : The term 'nvm' is not recognized as the name of a cmdlet, function, script file,
or operable program. Check the
spelling of the name, or if a path was included, verify that the path is correct and try
again.
```

```
At line:1 char:1
```

```
- nvm version
```

```
-  
...
```

```
+ CategoryInfo          : ObjectNotFound: (nvm:String) [],  
CommandNotFoundException  
+ FullyQualifiedErrorId : CommandNotFoundException  
...
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> nvm.cmd version
nvm.cmd : The term 'nvm.cmd' is not recognized as the name of a cmdlet, function,
script file, or operable program.
Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
```

```
At line:1 char:1
```

```
- nvm.cmd version
```

```
-  
...
```

```
+ CategoryInfo          : ObjectNotFound: (nvm.cmd:String) [],  
CommandNotFoundException  
+ FullyQualifiedErrorId : CommandNotFoundException  
...
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/10/2026, 5:36:10 PM



Thinking



Searching the web

github.com readme.com nodejs.org chromewebstore.google.com r2.nodejs.org makeareadme.com play.google.com
docs.readme.com en.wikipedia.org utrechtuniversity.github.io

• Searched 10 websites

✓ Worked for 19s

That just means **NVM is not installed**. Since you only want to test Node 25 once, I actually would **not install NVM** or uninstall your working Node 24 setup.

A safer option is to use the official **portable Node 25 ZIP** temporarily. Node provides a Windows x64 build of v25.9.0, so we can run it without changing your installed Node 24. ([Node.js](https://nodejs.org/en/download/package-manager/#windows))

1. Stay on your #108 branch

You are already here:

```
fix/108-frontend-localstorage-tests
```

Don't change any code.

2. Download portable Node 25

In PowerShell:

```
cd C:\Users\shayn\OneDrive\Desktop

Invoke-WebRequest `
  -Uri "https://nodejs.org/dist/v25.9.0/node-v25.9.0-win-x64.zip" `
  -OutFile "node25.zip"

Expand-Archive -Path "node25.zip" -DestinationPath "node25"
```

This uses the official Node distribution. Node 25.9.0 is an old/out-of-maintenance release now, which is fine because we're deliberately using it only to reproduce #108. ([Node.js](#))

3. Temporarily put Node 25 first in THIS PowerShell window

Run:

```
$oldPath = $env:Path
$env:Path = "C:\Users\shayn\OneDrive\Desktop\node25\node-v25.9.0-win-x64;$env:Path"
```

Now:

```
node -v
```

You should get:

```
v25.9.0
```

Also:

```
npm.cmd -v
```

4. Go back to your repo

```
cd C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool
```

Confirm once more:

```
node -v
git branch --show-current
```

You want:

```
v25.9.0
fix/108-frontend-localstorage-tests
```

5. Now test #108

Because contracts already exist/build in your current repo, first just try:

```
npm.cmd run test --workspace=@sport-analytics/frontend
```

This is the important run.

If our diagnosis is correct, this should now produce the:

```
TypeError: window.localStorage.clear is not a function
```

failures.

Don't change anything if it does. **Copy the full output and send it to me.** That becomes your proper "before" evidence.

6. Switching back to Node 24 is extremely easy

When we're finished testing, run:

```
$env:Path = $oldPath  
node -v
```

It should return:

```
v24.14.0
```

Or simply close that PowerShell window and open a new one - because we only changed the `PATH` for this one terminal session, your normal Node installation has not been replaced.

This approach is much safer for what we're doing than installing NVM, especially because NVM for Windows itself recommends removing existing standalone Node installations to avoid PATH/symlink conflicts. ([GitHub](#))

So go through **steps 2-5**, and send me the Node 25 test output.

Sources

[Node.js 25.9.0 \(Current\)](#) nodejs.org

[nvm-windows/README.md at master](#) github.com

8/10/2026, 5:36:12 PM



```
PS C:\Users\shayn\OneDrive\Desktop> cd C:\Users\shayn\OneDrive\Desktop\Sport-
Analytics-Tool
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> node -v

> > git branch --show-current
> > v25.9.0
> > fix/108-frontend-localstorage-tests
> > PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run test --
workspace=@sport-analytics/frontend

> @sport-analytics/frontend\@0.1.0 test
> vitest run

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/frontend

· src/App.test.tsx (13)
· src/pages/PublicBrowsePages.test.tsx (8)
✓ src/features/auth/AuthProvider.test.tsx (4)
✓ src/api/public-read.test.ts (3)
  > src/App.test.tsx (13)
    > public application and authentication interface (13)
      × keeps the landing page public and shows signed-out navigation
    \ [ beforeEach ]
      × renders the /create-account Google authentication page
    \ [ beforeEach ]
      × renders the /sign-in Google authentication page
    / [ beforeEach ]
      × starts managed Google OAuth with the callback return URL and shows progress
    / [ beforeEach ]
      × shows a safe Google authentication error without provider details
    / [ beforeEach ]
      × recognises an authenticated callback and opens the account page
    / [ beforeEach ]
      × handles OAuth cancellation without exposing provider details
    / [ beforeEach ]
      × handles OAuth provider errors without exposing provider details
    / [ beforeEach ]
      × shows a recoverable error when the callback produces no session
    / [ beforeEach ]
      × updates to signed-in navigation and displays only session identity information
    / [ beforeEach ]
      × signs out through Supabase, returns home, and restores signed-out navigation
    / [ beforeEach ]
      × shows a safe sign-out error and remains signed in when Supabase rejects the
      action
    / [ beforeEach ]
      × persists a manual theme selection
    / [ beforeEach ]
    ✓ src/api/client.test.ts (4)
    ✓ src/api/public-read.test.ts (3)
    > src/pages/PublicBrowsePages.test.tsx (8)
      > public browsing pages (8)
        × loads competitions anonymously and exposes their public records
      / [ beforeEach ]
        × shows a clear empty fixture state
      / [ beforeEach ]
        × shows a safe API failure and retries the request
      / [ beforeEach ]
        × keeps fixture filters and the next cursor in routed interface state
      / [ beforeEach ]
```

```
× opens a fixture and links its related public records
/[ beforeEach ]
× opens 'Wanderers' from its public collection
/[ beforeEach ]
× opens 'A Player' from its public collection
/[ beforeEach ]
× browses seasons and links them to their competitions
/[ beforeEach ]
✓ src/features/auth/AuthProvider.test.tsx (4)
```

Failed Tests 21

```
FAIL src/App.test.tsx > public application and authentication interface > keeps the
landing page public and shows signed-out navigation
FAIL src/App.test.tsx > public application and authentication interface > renders the
/create-account Google authentication page
FAIL src/App.test.tsx > public application and authentication interface > renders the
/sign-in Google authentication page
FAIL src/App.test.tsx > public application and authentication interface > starts
managed Google OAuth with the callback return URL and shows progress
FAIL src/App.test.tsx > public application and authentication interface > shows a safe
Google authentication error without provider details
FAIL src/App.test.tsx > public application and authentication interface > recognises
an authenticated callback and opens the account page
FAIL src/App.test.tsx > public application and authentication interface > handles
OAuth cancellation without exposing provider details
FAIL src/App.test.tsx > public application and authentication interface > handles
OAuth provider errors without exposing provider details
FAIL src/App.test.tsx > public application and authentication interface > shows a
recoverable error when the callback produces no session
FAIL src/App.test.tsx > public application and authentication interface > updates to
signed-in navigation and displays only session identity information
FAIL src/App.test.tsx > public application and authentication interface > signs out
through Supabase, returns home, and restores signed-out navigation
FAIL src/App.test.tsx > public application and authentication interface > shows a safe
sign-out error and remains signed in when Supabase rejects the action
FAIL src/App.test.tsx > public application and authentication interface > persists a
manual theme selection
TypeError: window.localStorage.clear is not a function
> src/App.test.tsx:96:25
94| describe('public application and authentication interface', () => {
95|   beforeEach(() => {
96|     window.localStorage.clear();
   |     ^
97|     document.documentElement.removeAttribute('data-theme');
98|     document.documentElement.removeAttribute('style');
```

```
FAIL src/pages/PublicBrowsePages.test.tsx > public browsing pages > loads
competitions anonymously and exposes their public records
FAIL src/pages/PublicBrowsePages.test.tsx > public browsing pages > shows a clear
empty fixture state
FAIL src/pages/PublicBrowsePages.test.tsx > public browsing pages > shows a safe
API failure and retries the request
FAIL src/pages/PublicBrowsePages.test.tsx > public browsing pages > keeps fixture
filters and the next cursor in routed interface state
FAIL src/pages/PublicBrowsePages.test.tsx > public browsing pages > opens a fixture
and links its related public records
FAIL src/pages/PublicBrowsePages.test.tsx > public browsing pages > opens
'Wanderers' from its public collection
```



```

FAIL src/pages/PublicBrowsePages.test.tsx > public browsing pages > opens 'A
Player' from its public collection
FAIL src/pages/PublicBrowsePages.test.tsx > public browsing pages > browses
seasons and links them to their competitions
TypeError: window.localStorage.clear is not a function
> src/pages/PublicBrowsePages.test.tsx:71:25
69| describe('public browsing pages', () => {
70|   beforeEach(() => {
71|     window.localStorage.clear();
   |     ^
72|     useSystemTheme();
73|   });

```

Test Files 2 failed | 3 passed (5)

Tests 21 failed | 11 passed (32)

Start at 17:39:57

Duration 1.95s (transform 314ms, setup 683ms, collect 1.08s, tests 192ms, environment 4.21s, prepare 900ms)

npm error Lifecycle script `test` failed with error:

npm error code 1

npm error path C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\frontend

npm error workspace @sport-analytics/frontend@0.1.0

npm error location C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\frontend

npm error command failed

npm error command C:\Windows\system32\cmd.exe /d /s /c vitest run

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>

8/10/2026, 5:40:23 PM



Thinking

Inspecting Repository Structure and Setup

- Inspected repository setup and investigated Node compatibility fixes

Searching the web

developer.mozilla.org

github.com

nodejs.org

w3schools.com

blog.logrocket.com

en.wikipedia.org

nodered.org

deno.com

nodesource.com

- Identifying the fix

I've reproduced all 21 failures and confirmed the mismatch with the older issue count. The common cause is Node 25's Web Storage behavior interacting with jsdom; I'm now choosing the smallest reliable environment-level fix.

- Identified the fix

Searching the web

- Searched 9 websites

Inspecting frontend TypeScript configuration

- Choosing a compatible fix

I'm favoring a targeted Vitest worker configuration that disables Node's Web Storage only on Node 25 or newer, preserving compatibility with the project's Node 20 baseline while letting jsdom provide its own browser storage.

- Chose a compatible fix

Searching Vitest global setup source

• Inspected TypeScript configuration and Vitest global window setup source

✓ Worked for 3m 40s

Yes - **you've now conclusively reproduced #108 yourself**. This is exactly what we needed.

Your evidence is now:

```
Node v24.14.0
→ frontend tests PASS

Node v25.9.0
→ 21 tests FAIL
→ TypeError: window.localStorage.clear is not a function
```

The issue originally said 17 because at `1f9a3aa` `App.test.tsx` had 9 affected tests. Current `main` has 13 there, so you now see **21 = 13 + 8** failures. It is still the exact same bug.

The root cause is also confirmed: Node 25 enabled Web Storage globally by default, and Node 25 documents that `localStorage` can be an empty object when no `--localstorage-file` is supplied. Vitest has an upstream issue describing exactly the Node-25-passes/Node-24-fails difference with jsdom. ([Node.js](#))

Now fix it

I recommend fixing the **Vitest environment**, not mocking `localStorage` in `setup.ts` and definitely not touching the tests.

Open:

```
apps/frontend/vite.config.ts
```

Currently you have:

```
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vitest/config';

export default defineConfig({
  plugins: [react()],
  optimizeDeps: {
    include: ['@sport-analytics/contracts'],
  },
  build: {
    commonjsOptions: {
      include: [/node_modules/, /packages[\\/]contracts/],
    },
  },
  test: {
    environment: 'jsdom',
    setupFiles: ['./src/test/setup.ts'],
    globals: true,
  },
});
```

Change it to:

```
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vitest/config';

const nodeMajorVersion = Number(process.versions.node.split('.')[0]);

export default defineConfig({
  plugins: [react()],
  optimizeDeps: {
    include: ['@sport-analytics/contracts'],
```

```

},
build: {
  commonjsOptions: {
    include: [/node_modules/, /packages[\\/]contracts/],
  },
},
test: {
  environment: 'jsdom',
  setupFiles: ['./src/test/setup.ts'],
  globals: true,
  poolOptions: {
    forks: {
      // Node 25+ exposes Web Storage globals that conflict with Vitest's
      // jsdom environment. Disable Node's implementation in test workers
      // so jsdom provides the browser Storage API.
      execArgv:
        nodeMajorVersion >= 25 ? ['--no-experimental-webstorage'] : [],
    },
  },
},
});

```

Why condition it on `>=25` ? Your repository supports Node 20, while `--no-experimental-webstorage` was added later. We therefore **must not pass that flag to Node 20**. Vitest 2 supports passing Node arguments specifically to its fork workers through `poolOptions.forks.execArgv`, and Vitest 2's default pool is `forks`. ([Vitest](#))

This is also better than replacing `localStorage` with a fake object: Node's Web Storage implementation is disabled **only inside affected test workers**, allowing the configured `jsdom` environment to supply the browser Storage API as intended. Node officially exposes `--no-experimental-webstorage` for this purpose. ([Node.js](#))

Test it while you are STILL on Node 25

After saving `vite.config.ts`, do:

```
node -v
```

Make sure it still says:

```
v25.9.0
```

Then:

```
npm.cmd run test --workspace=@sport-analytics/frontend
```

This is your crucial "after" test.

We want:

```

Test Files  5 passed (5)
Tests      32 passed (32)

```

If that happens, you've satisfied the most important part of #108:

```

BEFORE Node 25:
21 localStorage failures

AFTER Node 25:
32/32 pass

```

Then switch back to Node 24

If `$oldPath` is still defined in this PowerShell window:

```
$env:Path = $oldPath  
node -v
```

You want:

```
v24.14.0
```

If that doesn't work, simply close this PowerShell window and open a fresh one; your normal Node 24 installation was never removed.

Then run:

```
npm.cmd run test --workspace=@sport-analytics/frontend
```

That gives us regression evidence:

```
Node 25 → PASS  
Node 24 → PASS
```

Then we'll do lint/typecheck, check the diff, and prepare the commit/PR.

Don't commit yet. Make the `vite.config.ts` change and send me the Node 25 test output first.

Sources

[Global objects | Node.js v25.9.0 Documentation](#) nodejs.org

[Configuring Vitest | Vitest](#) v2.vitest.dev

8/10/2026, 5:40:25 PM



both pass!

8/10/2026, 5:45:40 PM



Thinking

- **Verifying the fix**

I'm ready to run the final checks now: lint, type-checking, formatting validation, and the frontend test on Node 24 and Node 25. I'll also confirm the diff and commit metadata are clean before finalizing.

- **Verified the fix**

✓ Worked for 14s

Excellent - that means #108 is properly reproduced **and fixed**.

You now have strong evidence:

```
BEFORE
Node v25.9.0
→ 21 tests failed
→ TypeError: window.localStorage.clear is not a function

AFTER FIX
Node v25.9.0
→ 32/32 tests passed

REGRESSION CHECK
Node v24.14.0
→ 32/32 tests passed
```

The issue originally mentioned **17 failures**, while current `main` gives **21**, because `App.test.tsx` has gained four tests since commit `1f9a3aa`. Same root failure.

1. Do the remaining checks on Node 24

Make sure you're back on Node 24:

```
node -v
```

Then:

```
npm.cmd run lint --workspace=@sport-analytics/frontend
npm.cmd run typecheck --workspace=@sport-analytics/frontend
```

Then:

```
git diff --check
git status
git diff
```

You ideally want the functional change to be just:

```
apps/frontend/vite.config.ts
```

2. Check the actual diff carefully

You should see essentially this addition:

```
const nodeMajorVersion = Number(process.versions.node.split('.')[0]);
```

and under `test` :

```
poolOptions: {
  forks: {
    // Node 25+ exposes Web Storage globals that conflict with Vitest's
    // jsdom environment. Disable Node's implementation in test workers
    // so jsdom provides the browser Storage API.
    execArgv:
      nodeMajorVersion >= 25 ? ['--no-experimental-webstorage'] : [],
  },
},
```

No changes to:

```
window.localStorage.clear();
```

That is important because the issue explicitly required identifying/fixing the cause rather than removing the cleanup calls.

3. Commit

Because ChatGPT generated code for this fix, the COMS3011A AI policy requires the AI code-generation attribution in the commit.
`ai_policy_20_jul.pdf`

I would use:

```
git add apps/frontend/vite.config.ts
git commit
```

Then:

```
fix(frontend): restore jsdom storage on Node 25

Disable Node 25's Web Storage implementation in Vitest fork
workers so jsdom provides the browser-compatible Storage API.

Refs #108

Assisted-by: ChatGPT-Web[GPT-5.6 Sol]
```

Then:

```
git push -u origin fix/108-frontend-localstorage-tests
```

4. PR description

Use something like:

```
## Related Issue

Fixes #108

## Purpose

Restore the frontend test suite on Node 25 without removing or bypassing
the existing `localStorage.clear()` test cleanup.

## Root Cause
```

Node 25 enables its Web Storage implementation by default.

Under the project's Vitest 2.1.9 `jsdom` test environment, this Node-provided `localStorage` can take precedence over jsdom's browser-compatible Storage implementation. The resulting object does not provide the expected `clear()` method, causing the affected suites to fail in `beforeEach`.

The problem does not reproduce on Node 24.

Changes

Updated `apps/frontend/vite.config.ts` so Vitest fork workers running on Node 25 or newer receive:

```
`--no-experimental-webstorage`
```

This disables Node's Web Storage implementation inside the test workers and allows the configured jsdom environment to provide the browser Storage API.

The flag is only supplied on Node 25+ so supported older Node versions are not passed an unsupported runtime option.

Verification

Before

Node v25.9.0:

- `App.test.tsx` failed in `beforeEach`
- `PublicBrowsePages.test.tsx` failed in `beforeEach`
- 21 current tests failed with:
`TypeError: window.localStorage.clear is not a function`

The issue originally reported 17 failures at commit `1f9a3aa`; additional tests have since been added to `App.test.tsx`.

After

Node v25.9.0:

- 5 test files passed
- 32 tests passed

Node v24.14.0:

- 5 test files passed
- 32 tests passed

No `localStorage.clear()` calls were removed or bypassed.

AI Declaration

The preceding Pull Request description was generated with the assistance of ChatGPT-Web[GPT-5.6 Sol].

This is now a **good fix for #108**, not a speculative one: you personally reproduced the failure on Node 25, applied the environment-level fix, and verified it on both Node 25 and Node 24.

8/10/2026, 5:45:52 PM



give me commit line with message, also, dont i need to first add AI evidence and this transcript b4 i commit?

8/10/2026, 5:47:21 PM



Thinking

Inspecting AI evidence files in repository zip

- **Preparing commit guidance**

I'm checking the requested files and commit message format. I'll keep the recommendation concise, include the transcript and register updates, and avoid claiming a commit hash before the commit exists.

- **Prepared commit guidance**

- **Inspected repository evidence files, issue records, commits, and Shayna AI usage patterns**

✓ Worked for 2m 4s

Yes - but there's one distinction.

For the **course AI policy**, the code commit itself must contain the `Assisted-by:` trailer because we used ChatGPT for code generation. `ai_policy_20_jul.pdf` Your repo's own AI-evidence process also expects the transcript and register entry, but those can be committed separately. In fact, that matches how your repo has already handled other work.

For #108 I'd keep it clean with **two commits**:

1. **Commit the actual fix**
2. **Commit the AI transcript + register entry**

First commit - the fix

Make sure only `vite.config.ts` is staged:

```
git add apps/frontend/vite.config.ts
```

Then use this exact one-line command:

```
git commit -m "fix(frontend): restore jsdom storage on Node 25" -m "Refs #108" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

That satisfies the required code-generation declaration.

Then add this transcript as evidence

Export this conversation and save it as:

```
evidence/ai/transcripts/shayna-unterslak/2026-08-10-issue-108-frontend-localstorage-tests.pdf
```

That matches your repository's established naming convention:

```
YYYY-MM-DD-issue-<number>-<short-description>.<extension>
```

Then add an entry to:

```
evidence/ai/registers/shayna-unterslak.csv
```

I suggest this row:

```
"2026-08-10","Shayna Underslak","ChatGPT Web","GPT-5.6 Sol","Debugging; code generation; testing guidance; technical guidance","Issue #108: diagnose and fix frontend localStorage test failures","Vitest configuration change disabling Node 25 Web Storage in test workers so jsdom provides the browser Storage API","Reproduced the reported localStorage.clear failure on Node v25.9.0; confirmed the same frontend suite passed on Node v24.14.0; applied the Vitest worker configuration fix; verified all 32 frontend tests pass on both Node v25.9.0 and Node v24.14.0 without removing the localStorage.clear calls","evidence/ai/transcripts/shayna-underslak/2026-08-10-issue-108-frontend-localstorage-tests.pdf; Issue #108; linked Pull Request"
```

Then:

```
git add evidence/ai/registers/shayna-underslak.csv
git add evidence/ai/transcripts/shayna-underslak/2026-08-10-issue-108-frontend-localstorage-tests.pdf
```

And make the evidence commit:

```
git commit -m "docs(evidence): record AI usage for issue 108" -m "Refs #108" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

Then check:

```
git status
git log --oneline -2
```

You should have something approximately like:

```
xxxxxxx docs(evidence): record AI usage for issue 108
xxxxxxx fix(frontend): restore jsdom storage on Node 25
```

So yes: make sure the transcript and AI register are added before you push/open the PR, but I recommend two commits rather than mixing the evidence PDF into the functional code commit. That is also consistent with the evidence structure already used in your repo.

8/10/2026, 5:47:23 PM



i need to format vite first, whats the command?

8/10/2026, 5:49:51 PM



Thinking



Inspecting frontend package scripts

- Inspected frontend package scripts in project and ZIP archive

✓ Worked for 49s

From the repo root, run:

```
npm.cmd run format
```

That runs Prettier across the whole repo.

If you only want to format the Vite config file, use:

```
npx.cmd prettier --write apps/frontend/vite.config.ts
```

I'd use the second one for #108 so you don't accidentally reformat unrelated files.

8/10/2026, 5:49:54 PM

